

# IEEE-CIS Fraud Detection

## Document 1 of 3 — Exploratory Data Analysis (EDA)

This document explains, cell-by-cell, everything the notebook does **before** any feature is engineered or any model is trained — i.e. the raw-data inspection phase. For each step it covers: what the code does, why that step exists, and what the actual output/finding was when it was run on the real IEEE-CIS dataset.

**Notebook section covered:** Cells 8–17 (environment/paths setup and Section 2 — "Load and merge the data").

**What comes next:** Document 2 covers Feature Engineering (Section 3). Document 3 covers Modeling, the Cascade architecture, and Evaluation Metrics (Sections 4–6.3).

## 1. What dataset are we working with?

This is the **IEEE-CIS Fraud Detection** dataset (a real, published Kaggle competition dataset from Vesta Corporation, a payment-processing company). Each row is one online credit-card transaction. The task is **binary classification**: predict `isFraud` — 1 if the transaction was fraudulent, 0 if legitimate.

The data ships as **four separate CSV files**, split into a labelled training portion and an unlabelled test portion that the notebook must predict for:

| File                               | What it contains                                                                                                    | Rows × Columns |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------|
| <code>train_transaction.csv</code> | Core transaction record (amount, card, product, etc.) + the <code>isFraud</code> label                              | 500,540 × 394  |
| <code>train_identity.csv</code>    | Device/network attributes (browser, OS, IP-derived signals) — only 14,238 transactions have a matching identity row | 14,238 × 41    |
| <code>test_transaction.csv</code>  | Same structure as <code>train_transaction</code> , no <code>isFraud</code> column (this is what we predict)         | 500,991 × 393  |
| <code>test_identity.csv</code>     | Same structure as <code>train_identity</code> , for the test period                                                 | 141,907 × 41   |

**Why the identity file is a separate, optional file:** device/network data isn't always captured (e.g. some checkout flows don't run the fingerprinting script). Vesta ships it separately rather than padding every row with blanks, and a **left join** is used later so transactions without identity data keep their transaction-level features and simply get NaN for identity-level ones. *Whether identity data exists at all for a transaction (i.e. whether the row is NaN or not) is itself treated as a signal*, not just a gap to fill — this is a genuine, well-documented pattern in fraud data: fraudsters using stripped-down or scripted checkout flows are less likely to trigger full device fingerprinting.

## 2. Step-by-step: what each EDA cell does and why

### 2.1 Environment & path setup (Cells 8–11)

*What the code does*

Before touching any data, the notebook checks the current working directory, locates the dataset folder (a local Windows path in this case, stored in the environment variable `IEEE_DATA_DIR`), and confirms that `train_transaction.csv` and `train_identity.csv` actually exist on disk before proceeding.

*Why this step exists*

This is defensive engineering, not analysis — the notebook is written to also run unchanged on Kaggle (where the path is `/kaggle/input/...`) or on any teammate's machine, as long as `IEEE_DATA_DIR` is set correctly. Failing fast here with a clear `FileNotFoundError` avoids a confusing crash 20 minutes into a long training run.

## 2.2 Global setup — imports, GPU detection, seeding (Cell 12)

*What the code does*

- Sets a fixed random **SEED = 42** everywhere (Python's random, NumPy, and environment hash seed) so re-running the notebook reproduces the same folds, same model initializations, and same results.
- Probes whether a usable GPU is present (via `nvidia-smi` and a 1-round LightGBM training probe) and configures device settings for LightGBM, XGBoost, and CatBoost accordingly, falling back to CPU automatically if no GPU is available or if the LightGBM build doesn't support GPU.
- Resolves `DATA_DIR` (where to read from) and `OUT_DIR` (where to write outputs like submission files and diagnostics).

*Why this step exists*

**Reproducibility** and **portability**. A fixed seed means the exact same cross-validation folds, the exact same model weight initializations, and (on CPU) near-identical results every time the notebook reruns — critical for being able to trust that a metric change later came from a real code change, not random noise. GPU auto-detection means the same notebook runs correctly whether it's on a laptop with no GPU or a Kaggle T4/P100 session.

## 2.3 Loading and merging the four files (Cells 16–17)

*What the code does*

Reads all four CSVs with `pandas.read_csv`, then performs a **left join** of each transaction file with its corresponding identity file on `TransactionID`. It also runs a memory-reduction pass (`reduce_mem_usage`) that downcasts every numeric column to the smallest dtype that safely holds its value range (e.g. `float64` → `float32`, `int64` → `int8/16/32` where the range allows).

*Why this step exists*

- **Left join, not inner join:** an inner join would silently drop the ~76% of transactions that have no identity data, which would throw away most of the training set. A left join keeps every transaction and fills identity columns with NaN when there's no match — correct given identity is optional signal, not core data.
- **Memory reduction:** the merged frame is roughly 1.1 million rows × 430+ columns. At default `float64/int64` dtypes this can exceed available RAM on a single Kaggle session or a laptop. Downcasting is a purely computational optimization — it does not change any actual values, only how many bytes each one occupies in memory.

- **Column name mismatch handled explicitly:** the test identity file uses hyphens (id-01) while the train identity file uses underscores (id\_01) — a real inconsistency in how Kaggle packaged this competition's files. The code renames test's columns to match train's before anything else happens; skipping this would silently create ~38 duplicate/misaligned columns between train and test.

*Actual output when run*

```
train_transaction (590,540, 394)   train_identity (144,233, 41)
test_transaction  (506,691, 393)   test_identity  (141,907, 41)
train (590,540, 434)   test (506,691, 433)
memory 1984.2 MB -> 1073.5 MB (45.9% saved)   [train]
memory 1700.3 MB -> 922.3 MB (45.8% saved)    [test]
```

The memory-reduction step alone nearly halved RAM usage on both frames — this is what makes it practical to hold the full dataset (train + test combined, ~1.1M rows) in memory throughout feature engineering without running out of RAM.

### 3. Orientation pass — what the data actually looks like

Before any feature engineering, the notebook runs one diagnostic cell to answer four questions that determine every downstream design decision: how imbalanced is the target, is there a time gap between train and test, how much identity data is actually present, and which columns are mostly missing.

*Actual output when run*

```
rows          train=590,540  test=506,691
fraud rate    3.4990%  (20,663 positives)
TransactionDT train [86,400 .. 15,811,131]
              test  [18,403,224 .. 34,214,345]
gap between them 30.0 days

identity coverage  train=23.8%  test=27.0%

columns >90% missing: 12 of 434
id_24  0.9920   id_25  0.9913   id_07  0.9913   id_08  0.9913
id_21  0.9913   id_26  0.9913   id_27  0.9912   id_23  0.9912
```

#### 3.1 Finding — Severe class imbalance (this is your 3% question)

Only **3.499% of transactions are fraud** (20,663 out of 590,540). This single number drives almost every later design decision in the notebook:

- **Metric choice:** plain accuracy is useless here — a model predicting "not fraud" for every single transaction would score 96.5% accuracy while catching zero fraud. This is exactly why the entire notebook is built around **ROC-AUC** and **PR-AUC** instead (explained in full in Document 3), which are not distorted by class imbalance the same way.
- **Validation strategy:** with only ~20,663 positive examples spread across a 3-fold time-based split, each fold's validation set will have a similarly low, sometimes uneven fraud rate — worth watching for when judging whether one fold's score is trustworthy (see Section 4 below and Document 3, Section 4).
- **Stage-1 recall target in the cascade (98%):** because fraud is rare and expensive to miss, the cascade's screening stage is deliberately tuned to almost never let a fraud case through undetected, even at the

cost of flagging many legitimate transactions for further review. Full explanation in Document 3, Section 6.

### 3.2 Finding — A real, 30-day time gap between train and test

TransactionDT is a raw seconds-since-an-arbitrary-origin counter, not a calendar date. Train covers roughly the first ~183 days, and test begins exactly 30 days after train ends — **test is entirely in the future of train**, with no overlap.

**Why this matters more than almost anything else in this project:** this single fact is the reason the notebook uses **time-aware, month-grouped cross-validation** instead of a random K-fold split (fully explained in Section 4 below). A random split would let the model "see" cards, devices, and patterns from the same time period on both sides of the split, which inflates validation scores in a way that would not hold up on the real, future-only test set — and would not reflect how a fraud model actually gets deployed in production (always predicting on transactions that happen after training data was collected).

### 3.3 Finding — Identity data is sparse and asymmetric (23.8% vs 27.0%)

Only 23.8% of train rows and 27.0% of test rows have a matching identity record. This confirms the left-join decision above was correct — an inner join would have discarded the majority of the dataset. It also means every identity-derived feature engineered later must tolerate a large amount of missingness by design, not as an afterthought.

### 3.4 Finding — 12 columns are >90% missing

A dozen identity columns (mostly device-fingerprinting fields like id\_24, id\_07, id\_08) are missing in over 99% of rows. These are flagged here for awareness but are **not dropped yet** — the actual drop decision happens later in feature engineering, using a slightly different, sample-based near-empty/near-constant check (covered in Document 2). This EDA pass is purely diagnostic: it tells the team what to expect before the feature-engineering code makes any irreversible decisions.

## 4. How the data is split — train / validation / test, explained simply

This is worth explaining clearly since it's a common point of confusion. There are **three** distinct roles data plays in this project, and it's important not to mix them up:

- **Train set** (590,540 rows, has labels): the data the models actually learn from. Every model (LightGBM, XGBoost, CatBoost) sees this data's features *and* its true isFraud labels during fitting.
- **Validation set** (carved out of the train set, has labels): a portion of the train set temporarily held back from a model during its training, so we can check how well it predicts on data it did *not* memorize. This is what all the OOF (out-of-fold) scores throughout the notebook are measuring.

- **Test set** (506,691 rows, *no labels provided*): the actual unseen future data the whole project is ultimately judged on. No model ever trains on this, and — critically — **no threshold or hyperparameter in this notebook is ever selected by looking at test-set outcomes**, because the labels for it aren't even available locally (this mirrors how Kaggle grades submissions, and how a real production model would be judged on genuinely future transactions).

## 4.1 Why a random 80/20 split would be the wrong choice here

The obvious, common approach — shuffle all rows randomly and hold out 20% as validation — would badly overstate this model's real accuracy on this dataset. Recall from Section 3.2: test is a full 30 days in the future of train. A random split ignores that entirely and can put transactions from the same card, same device, or same short time window on *both* sides of the split. The model can then partially "recognize" validation rows from patterns it saw in near-identical training rows moments earlier — producing a validation score that looks great but does not represent genuine future-prediction skill.

## 4.2 What the notebook does instead: GroupKFold on calendar month

*What the code does*

TransactionDT is converted into a calendar-month bucket (DT\_M, built purely for this purpose and dropped before training — see Document 2). GroupKFold(*n\_splits*=3) is then used with month as the group: every transaction from a given month is guaranteed to land entirely in either the training side or the validation side of a fold — never split across both.

*Actual output when run*

```
GroupKFold(month) x3
fold 0: train=369,564  valid=220,976  valid fraud=2.886%  months=[12, 16]
fold 1: train=402,887  valid=187,653  valid fraud=3.979%  months=[14, 15]
fold 2: train=408,629  valid=181,911  valid fraud=3.749%  months=[13, 17]
```

**What "OOF" (out-of-fold) means, concretely:** across the 3 folds above, every single training row gets predicted exactly once, by a model that was trained on the *other two* folds' months and had never seen that row (or its month) during training. Collecting all three folds' validation predictions together gives one full-length "OOF" prediction array covering the entire train set, built without any row ever being predicted by a model that trained on it. This is the array every AUC/PR-AUC/precision/recall number in Document 3 is computed from — it is the closest honest proxy available for "how will this model do on genuinely unseen future data", short of actually having the real test labels.

Notice also that fold validation fraud rates genuinely vary (2.886% / 3.979% / 3.749%) — another visible consequence of both the class imbalance (Section 3.1) and real seasonal variation in fraud rate by month. This is exactly why the notebook reports both a pooled OOF score and a per-fold mean/std (Document 3, Section 4) — a single number can hide how much that score wobbles month to month.

## 5. Summary — what this EDA phase established

- The dataset is a real ~1.1M-row transaction log, 3.499% fraud, split across 4 files that get left-joined into one working frame per train/test.

- Class imbalance (3.5% fraud) is severe enough that accuracy is not a usable metric — this decision carries into every later stage (see Document 3).
- Test is strictly 30 days in the future of train, with zero overlap — this is why time-aware, month-grouped validation is used instead of a random split (Section 4).
- Identity data is optional and sparse (~24-27% coverage) — a left join preserves every row, and missingness itself is treated as potential signal, not just a gap.
- A dozen columns are near-empty and flagged here for awareness; the actual drop decision is made later, during feature engineering (Document 2).

---

*End of Document 1 — continue with Document 2: Feature Engineering*